Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации
Ордена Трудового Красного Знамени Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский технический университет связи и информатики»
(МТУСИ)

Кафедра информатики

Лабораторная работа №3

по дисциплине

Информационные технологии и программирование

на тему

Разработка алгоритмов нахождения эйлеровых графов и их путей

Выполнила: студентка группы БКС2505 Зотова.З.В

Проверил: Халабия Р.Ф.

Москва 2026

1. Цели и задачи.

Изучить методы нахождения и построения эйлеровых графов и их путей. Освоить на практике проверку графа на эйлеровость, построение эйлерова цикла и эйлеровой цепи алгоритмом Флёри, а также способ изменения графа, при котором построение эйлерова цикла становится возможным.

Маршрутом в графе G(V, E) называется чередующаяся последовательность вершин и рёбер v0, e1, …, en, vn, в которой любые два соседних элемента инцидентны. Если v0 = vn, маршрут замкнут, иначе открыт. Если все рёбра маршрута различны, он называется цепью, а замкнутая цепь называется циклом.

Эйлеров цикл – цикл, содержащий все рёбра графа ровно по одному разу; граф, имеющий эйлеров цикл, называется **эйлеровым**. **Эйлерова цепь** – цепь, содержащая все рёбра графа ровно по одному разу; граф, у которого есть эйлерова цепь, но нет эйлерова цикла, называется **полуэйлеровым**. Эйлеровым может быть только связный граф (изолированные вершины не учитываются).

Теорема Эйлера. Связный неориентированный граф является эйлеровым тогда и только тогда, когда степени всех его вершин чётные. Связный граф является полуэйлеровым тогда и только тогда, когда в нём ровно две вершины нечётной степени; эйлерова цепь начинается в одной из этих вершин и заканчивается в другой.

Алгоритм Флёри строит эйлеров цикл (цепь): начиная с некоторой вершины, на каждом шаге выбирается ребро, инцидентное текущей вершине, оно удаляется из текущего графа и вносится в список, а текущей становится вторая вершина ребра. Ограничение: если степень текущей вершины больше 1, нельзя выбирать ребро, удаление которого увеличит число компонент связности (т. е. ребро-мост). Работа заканчивается, когда в текущем графе не останется рёбер.

2. Задание.

2.1 Постановка задачи.

Написать программу нахождения и построения эйлеровых графов и их путей согласно варианту 4. Программа должна определить, является ли граф эйлеровым, и соответственно выдать эйлеров цикл или эйлерову цепь. Если граф не является эйлеровым, программа должна изменить граф так, чтобы построение эйлерова цикла стало возможным. Граф варианта 4 показан на рисунке 1.

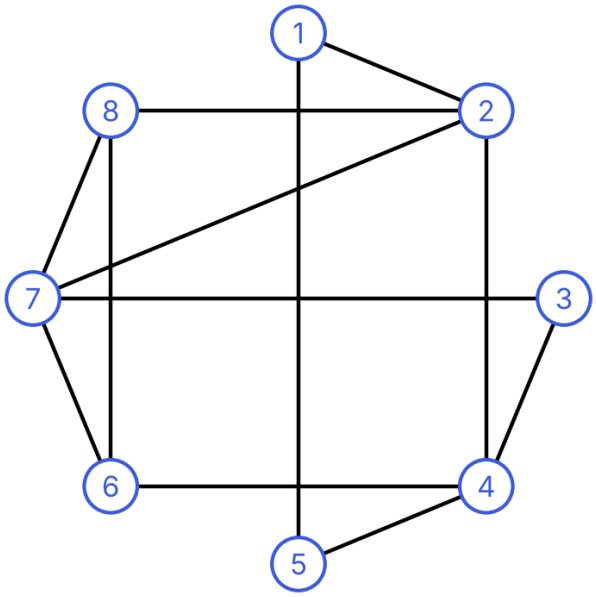


Рисунок 1 – граф варианта 4.

Граф содержит 8 вершин и 12 рёбер: v1–v2, v1–v5, v2–v4, v2–v7, v2–v8, v3–v4, v3–v7, v4–v5, v4–v6, v6–v7, v6–v8, v7–v8. Степени вершин приведены в таблице 1.

Таблица 1 – степени вершин графа варианта 4.

Вершина	**v1**	**v2**	**v3**	**v4**	**v5**	**v6**	**v7**	**v8**
Степень d(v)	2	4	2	4	2	3	4	3

Граф связный, но вершины v6 и v8 имеют нечётную степень, поэтому граф **не является эйлеровым**, а является **полуэйлеровым**: в нём существует эйлерова цепь из v6 в v8. Чтобы сделать граф эйлеровым, нужно сделать степени v6 и v8 чётными. Так как вершины v6 и v8 уже смежны, добавление ребра v6–v8 дало бы кратное ребро, поэтому в граф добавляется новая вершина v9 и рёбра v6–v9 и v9–v8. После этого степени всех вершин становятся чётными и эйлеров цикл существует.

2.2 Алгоритм решения задания.

Используемые переменные:

n – количество вершин; m – количество рёбер; adj – матрица смежности (adj[u][v] – число рёбер между u и v); choice – способ задания графа (1 – граф варианта 4, 0 – ввод с клавиатуры); odd – список вершин нечётной степени; visited – отметки посещения вершин при обходе в глубину; start – стартовая вершина алгоритма Флёри; cur – текущая вершина; next – следующая вершина (используется только в цикле); path – список вершин построенного цикла (цепи); saved – копия матрицы смежности; before, after – число вершин, достижимых из текущей до и после удаления ребра (проверка на мост); u, v, k, i – счётчики и номера вершин (используются только в циклах).

1. Начало

2. Ввод choice

3. Выбор по значению choice, выполняем один из шагов 3.1–3.3

- **3.1.** Если choice = 1 То n = 8, m = 12, заполнить adj рёбрами варианта 4
- **3.2.** Если choice = 0 То ввести n, m и m пар вершин, заполнить adj
- **3.3.** Иначе вывести «Неверный выбор» и завершить программу

4. Вывести матрицу смежности adj

5. Цикл для v от 1 до n, выполняем шаги 5.1–5.3

- **5.1.** Вычислить d(v) как сумму элементов строки v матрицы adj
- **5.2.** Вывести d(v)

5.3. Если d(v) нечётна То добавить v в список odd

6. Выполнить обход в глубину из первой вершины ненулевой степени

7. Если есть вершина ненулевой степени, не посещённая обходом, То вывести «Граф несвязный» и перейти к шагу 15

8. Если список odd пуст То вывести «Граф является эйлеровым», построить эйлеров цикл (шаг 13), вывести его и перейти к шагу 15

9. Если в odd ровно две вершины То вывести «Граф полуэйлеров», построить эйлерову цепь из вершины odd[0] (шаг 13) и вывести её Иначе вывести «Граф не является ни эйлеровым, ни полуэйлеровым»

10. Цикл по парам (u, v) вершин из списка odd, выполняем шаг 10.1

10.1. Если adj[u][v] = 0 То добавить ребро u – v Иначе n = n + 1, добавить вершину vn и рёбра u – vn, vn – v

11. Вывести матрицу смежности и степени вершин изменённого графа

12. Построить эйлеров цикл изменённого графа (шаг 13), вывести его; перейти к шагу 15

13. Алгоритм Флёри для вершины start, выполняем шаги 13.1–13.4

13.1. Сохранить adj в saved

13.2. Задать cur = start, внести start в path

13.3. Пока d(cur) > 0, выполняем шаги 13.3.1–13.3.4

13.3.1. Цикл для u от 1 до n: выбрать первую вершину u, для которой adj[cur][u] > 0 и (d(cur) = 1 или ребро cur – u не является мостом, шаг 14)

13.3.2. Удалить ребро cur – u из adj

13.3.3. Внести u в path

13.3.4. Задать cur = u

13.4. Восстановить adj из saved и вернуть path

14. Проверка ребра u – v на мост, выполняем шаги 14.1–14.4

14.1. Подсчитать before – число вершин, достижимых из u

14.2. Временно удалить ребро u – v и подсчитать after – число вершин, достижимых из u

14.3. Вернуть ребро u – v

14.4. Если after < before То ребро является мостом Иначе не является

15. Конец

2.3 Контрольный тест со скриншотом работы программы.

Входные данные: граф варианта 4 (choice = 1). Ручной расчёт: степени вершин – в таблице 1, вершин нечётной степени две (v6 и v8), следовательно, строится эйлерова цепь из v6. Пошаговая работа алгоритма Флёри приведена в таблице 2: на каждом шаге выбирается ребро к вершине с наименьшим номером, если оно не является мостом.

Таблица 2 – построение эйлеровой цепи алгоритмом Флёри.

Шаг	**Текущая вершина**	**Смежные вершины**	**Мосты**	**Выбранное ребро**
1	v6	v4, v7, v8	–	v6 – v4
2	v4	v2, v3, v5	–	v4 – v2
3	v2	v1, v7, v8	–	v2 – v1
4	v1	v5	v1 – v5 (единственное)	v1 – v5
5	v5	v4	v5 – v4 (единственное)	v5 – v4
6	v4	v3	v4 – v3 (единственное)	v4 – v3
7	v3	v7	v3 – v7 (единственное)	v3 – v7
8	v7	v2, v6, v8	–	v7 – v2
9	v2	v8	v2 – v8 (единственное)	v2 – v8
10	v8	v6, v7	–	v8 – v6
11	v6	v7	v6 – v7 (единственное)	v6 – v7
12	v7	v8	v7 – v8 (единственное)	v7 – v8

Получена эйлерова цепь v6 – v4 – v2 – v1 – v5 – v4 – v3 – v7 – v2 – v8 – v6 – v7 – v8, содержащая все 12 рёбер ровно по одному разу (рисунок 2, цифрами показан порядок обхода рёбер).

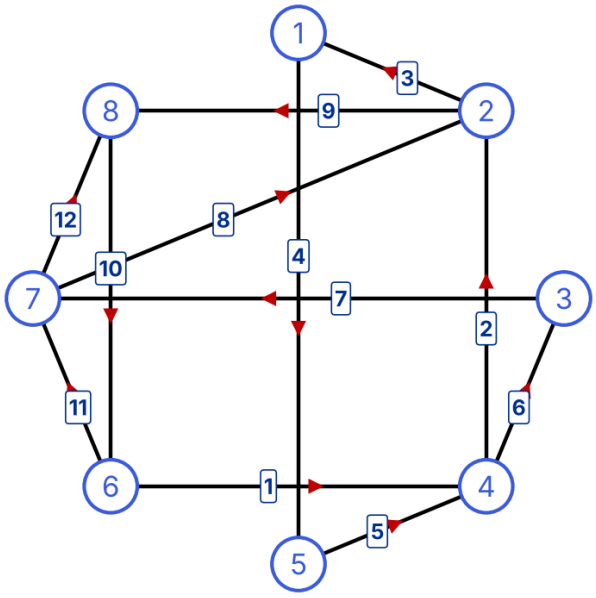


Рисунок 2 – эйлерова цепь графа варианта 4.

Далее граф изменяется: добавляется вершина v9 и рёбра v6 – v9, v9 – v8. Теперь степени всех вершин чётные (d(v6) = d(v8) = 4, d(v9) = 2), и алгоритм Флёри из вершины v1 строит эйлеров цикл из 14 рёбер. На шаге 3 (вершина v4) ребро v4 – v5 является мостом, поэтому выбирается v4 – v3; на шаге 8 (вершина v6) мостом является v6 – v4, поэтому выбирается v6 – v7. Итоговый цикл: v1 – v2 – v4 – v3 – v7 – v2 – v8 – v6 – v7 – v8 – v9 – v6 – v4 – v5 – v1 (рисунок 3).

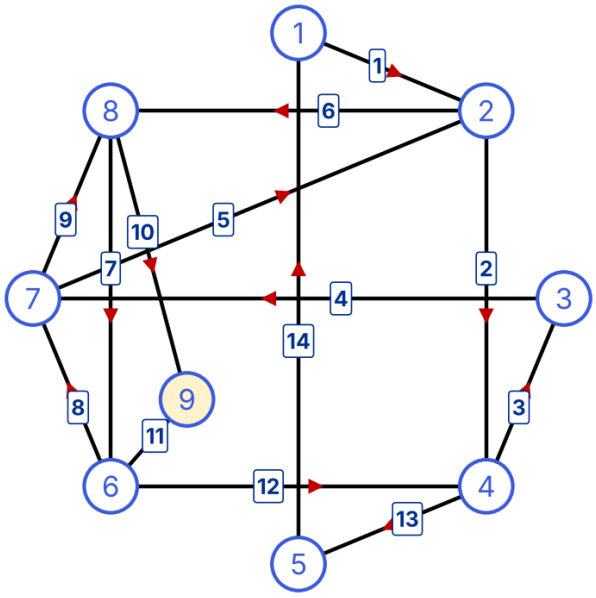


Рисунок 3 – изменённый граф (добавлена вершина v9) и его эйлеров цикл.

Результат работы программы показан на рисунке 4; он полностью совпадает с ручным расчётом.



Рисунок 4 – результат работы программы для графа варианта 4.

2.4 Выводы по работе.

В ходе выполнения работы разработана программа на C++, которая по матрице смежности определяет степени вершин, проверяет связность графа и по теореме Эйлера определяет, является ли граф эйлеровым, полуэйлеровым или ни тем, ни другим. Эйлеров цикл и эйлерова цепь строятся алгоритмом Флёри с проверкой ребра на мост, что гарантирует обход всех рёбер ровно по одному разу.

Граф варианта 4 оказался полуэйлеровым: вершины v6 и v8 имеют нечётную степень 3. Программа построила эйлерову цепь из v6 в v8 длиной 12 рёбер, а затем изменила граф, добавив вершину v9 и рёбра v6 – v9, v9 – v8 (кратное ребро v6 – v8 не добавлялось), после чего построила эйлеров цикл длиной 14 рёбер. Результаты программы совпали с ручным расчётом.

Сложность алгоритма Флёри с проверкой мостов обходом в глубину по матрице смежности составляет $O(m \cdot n^2)$, что вполне приемлемо для графов небольшого размера. Программа также позволяет ввести произвольный граф с клавиатуры, корректно обрабатывает несвязные графы и графы с числом нечётных вершин больше двух.

2.5 Исходный код программы.

Ниже приведён код программы, с помощью которого проводился контрольный тест (рисунок 5).

```
#include <iostream>
#include <vector>
#include <iomanip>
#include <cstdlib>

using namespace std;

int n;                    // количество вершин графа
vector<vector<int>> adj;      // матрица смежности (adj[u][v] - число рёбер между u и v)

void addEdge(int u, int v) // функция добавления ребра u - v
{
   adj[u][v]++; // увеличиваем число рёбер u - v
   adj[v][u]++; // граф неориентированный, поэтому и v - u
}

void removeEdge(int u, int v) // функция удаления ребра u - v
{
   adj[u][v]--; // уменьшаем число рёбер u - v
   adj[v][u]--; // и v - u
}
```

```
int degree(int v) // функция вычисления степени вершины
{
   int d = 0; // счётчик рёбер
   for (int u = 1; u <= n; u++) // перебираем все вершины
   {
      d += adj[v][u]; // прибавляем количество рёбер v - u
   }
   return d; // возвращаем степень вершины
}

void dfs(int v, vector<bool>& visited) // обход графа в глубину
{
   visited[v] = true; // отмечаем вершину как посещённую
   for (int u = 1; u <= n; u++) // перебираем всех соседей
   {
      if (adj[v][u] > 0 && !visited[u]) // если есть ребро и сосед ещё не
посещён
      {
         dfs(u, visited); // переходим в соседа
      }
   }
}

int countReachable(int v) // число вершин, достижимых из v
{
   vector<bool> visited(n + 1, false); // массив отметок посещения
   dfs(v, visited); // обходим граф из вершины v
   int cnt = 0; // счётчик достижимых вершин
   for (int u = 1; u <= n; u++) // перебираем все вершины
```

```
    {
        if (visited[u])
        {
            cnt++; // считаем посещённые
        }
    }
    return cnt; // возвращаем результат
}

bool isConnected() // проверка связности графа (без учёта изолированных вершин)
{
    int start = 0; // вершина, с которой начинаем обход
    for (int v = 1; v <= n; v++) // ищем первую вершину с ненулевой степенью
    {
        if (degree(v) > 0)
        {
            start = v;
            break;
        }
    }
    if (start == 0)
    {
        return true; // в графе нет рёбер - считаем связным
    }
    vector<bool> visited(n + 1, false); // массив отметок посещения
    dfs(start, visited); // обходим граф
    for (int v = 1; v <= n; v++) // проверяем все вершины
```

```
    {
        if (degree(v) > 0 && !visited[v])
        {
            return false; // вершина с рёбрами не достигнута
        }
    }
    return true; // все вершины с рёбрами достигнуты - граф связный
}

bool isBridge(int u, int v) // проверка: является ли ребро u - v мостом
{
    if (adj[u][v] > 1)
    {
        return false; // кратное ребро не может быть мостом
    }
    int before = countReachable(u); // число достижимых вершин до удаления ребра
    removeEdge(u, v); // временно удаляем ребро
    int after = countReachable(u); // число достижимых вершин после удаления
    addEdge(u, v); // возвращаем ребро на место
    return after < before; // если стало меньше - граф распался, ребро является мостом
}

vector<int> fleury(int start) // алгоритм Флёри построения эйлерова пути (цикла)
{
```

```
    vector<vector<int>> saved = adj; // сохраняем граф, так как рёбра будут удаляться
    vector<int> path; // список вершин пути
    path.push_back(start); // начинаем с заданной вершины
    int cur = start; // текущая вершина
    while (degree(cur) > 0) // пока у текущей вершины есть рёбра
    {
        int next = 0; // следующая вершина
        for (int u = 1; u <= n; u++) // перебираем соседей по возрастанию номера
        {
            if (adj[cur][u] == 0)
            {
                continue; // ребра нет - пропускаем
            }
            if (degree(cur) == 1 || !isBridge(cur, u)) // ограничение: мост выбираем только если нет других рёбер
            {
                next = u; // нашли подходящее ребро
                break; // выходим из цикла
            }
        }
        removeEdge(cur, next); // удаляем пройденное ребро из текущего графа
        path.push_back(next); // вносим вершину в список
        cur = next; // делаем её текущей
    }
    adj = saved; // восстанавливаем исходный граф
    return path; // возвращаем построенный путь
```

```
}

int firstVertex() // первая вершина с ненулевой степенью
{
   for (int v = 1; v <= n; v++) // перебираем вершины по возрастанию
   {
      if (degree(v) > 0)
      {
         return v; // нашли вершину, у которой есть рёбра
      }
   }
   return 1; // рёбер нет - возвращаем вершину 1
}

void printMatrix() // вывод матрицы смежности
{
   cout << "    ";
   for (int v = 1; v <= n; v++)
   {
      cout << setw(3) << v; // номера столбцов
   }
   cout << endl;
   for (int u = 1; u <= n; u++) // строки матрицы
   {
      cout << setw(3) << u << " "; // номер строки
      for (int v = 1; v <= n; v++)
      {
         cout << setw(3) << adj[u][v]; // элементы строки
      }
```

```
        cout << endl;
    }
}

void printDegrees(vector<int>& odd) // вывод степеней вершин и поиск нечётных
{
    odd.clear(); // очищаем список вершин нечётной степени
    cout << "Степени вершин:" << endl;
    for (int v = 1; v <= n; v++) // перебираем вершины
    {
        int d = degree(v); // степень вершины v
        cout << "  d(v" << v << ") = " << d << (d % 2 ? "  (нечётная)" : "") << endl;
        if (d % 2 != 0)
        {
            odd.push_back(v); // запоминаем вершину нечётной степени
        }
    }
}

void printPath(const vector<int>& path) // вывод пути
{
    for (int i = 0; i < (int)path.size(); i++) // перебираем вершины пути
    {
        cout << "v" << path[i]; // выводим вершину
        if (i + 1 < (int)path.size())
        {
            cout << " - "; // разделитель между вершинами
```

```
        }
    }
    cout << endl;
    cout << "Количество пройденных рёбер: " << path.size() - 1 << endl;
}

int main() // главная функция
{
    setlocale(LC_ALL, "rus");

    int m; // количество рёбер
    int choice; // выбор способа задания графа
    cout << "Использовать граф варианта 4 (1 - да, 0 - ввести свой граф): ";
    cin >> choice;

    switch (choice) // выбираем способ задания графа
    {
    case 1: // граф варианта 4
    {
        n = 8; // 8 вершин
        adj.assign(n + 1, vector<int>(n + 1, 0)); // создаём пустую матрицу
смежности
        int edges[12][2] = { { 1,2 },{ 1,5 },{ 2,4 },{ 2,7 },{ 2,8 },{ 3,4 },
                             { 3,7 },{ 4,5 },{ 4,6 },{ 6,7 },{ 6,8 },{ 7,8 } }; // рёбра
варианта 4
        m = 12; // количество рёбер
        for (int i = 0; i < m; i++) // перебираем все рёбра
        {
```

```
            addEdge(edges[i][0], edges[i][1]); // добавляем ребро
        }
        break; // выходим из switch
    }
    case 0: // ввод графа с клавиатуры
    {
        cout << "Введите количество вершин и рёбер: ";
        cin >> n >> m;
        adj.assign(n + 1, vector<int>(n + 1, 0)); // создаём пустую матрицу
смежности
        cout << "Введите рёбра (пары вершин от 1 до " << n << "):" << endl;
        for (int i = 0; i < m; i++) // читаем m рёбер
        {
            int u, v;
            cin >> u >> v;
            addEdge(u, v); // добавляем ребро
        }
        break; // выходим из switch
    }
    default: // введено что-то кроме 1 и 0
    {
        cout << "Неверный выбор. Введите 1 или 0." << endl;
        system("PAUSE"); // пауза перед закрытием окна консоли
        return 0; // завершаем программу
    }
    }

    cout << "\nМатрица смежности графа:" << endl;
    printMatrix(); // выводим матрицу смежности
```

```
    vector<int> odd; // вершины нечётной степени
    printDegrees(odd); // выводим степени и находим нечётные вершины

    if (!isConnected()) // эйлеровым может быть только связный граф
    {
        cout << "\nГраф несвязный - эйлеров цикл и эйлерова цепь
невозможны." << endl;
        system("PAUSE"); // пауза перед закрытием окна консоли
        return 0;
    }
    cout << "\nГраф связный." << endl;
    cout << "Количество вершин нечётной степени: " << odd.size() << endl;

    if (odd.empty()) // все степени чётные
    {
        cout << "\nГраф является эйлеровым." << endl;
        cout << "Эйлеров цикл: ";
        printPath(fleury(firstVertex())); // строим цикл из первой вершины
    }
    else
    {
        if (odd.size() == 2) // ровно две вершины нечётной степени
        {
            cout << "\nГраф не является эйлеровым, но является
полуэйлеровым." << endl;
            cout << "Эйлерова цепь (из v" << odd[0] << " в v" << odd[1] << "): ";
```

```
        printPath(fleury(odd[0])); // цепь начинается в одной нечётной
вершине
    }
    else // нечётных вершин больше двух
    {
        cout << "\nГраф не является ни эйлеровым, ни полуэйлеровым." <<
endl;
    }

    // Изменение графа: попарно соединяем вершины нечётной степени
    cout << "\nИзменение графа для построения эйлерова цикла:" << endl;
    for (int i = 0; i + 1 < (int)odd.size(); i += 2) // берём нечётные вершины
парами
    {
        int u = odd[i], v = odd[i + 1]; // пара вершин нечётной степени
        if (adj[u][v] == 0) // если вершины не смежны
        {
            addEdge(u, v); // добавляем ребро между ними
            cout << "  добавлено ребро v" << u << " - v" << v << endl;
        }
        else // вершины уже смежны - добавляем новую вершину, чтобы не
было кратных рёбер
        {
            n++; // новая вершина с номером n
            for (int k = 1; k < n; k++)
            {
                adj[k].push_back(0); // расширяем строки матрицы
            }
```

```
            adj.push_back(vector<int>(n + 1, 0)); // добавляем новую строку
            addEdge(u, n); // ребро u - новая вершина
            addEdge(n, v); // ребро новая вершина - v
            cout << "  добавлена вершина v" << n << " и рёбра v" << u << " - v"
<< n
                << ", v" << n << " - v" << v << endl;
        }
    }

    cout << "\nМатрица смежности изменённого графа:" << endl;
    printMatrix(); // выводим новую матрицу
    printDegrees(odd); // проверяем степени

    if (odd.empty() && isConnected()) // теперь все степени чётные
    {
        cout << "\nИзменённый граф является эйлеровым." << endl;
        cout << "Эйлеров цикл: ";
        printPath(fleury(firstVertex())); // строим эйлеров цикл
    }
}

system("PAUSE"); // пауза перед закрытием окна консоли
return 0; // успешное завершение программы
}
```

Рисунок 5 – исходный код программы.